

Device Integration Plugin für zenLAB

Kompaktes Implementierungskonzept für die Abstimmung mit dem Betreuer

Punkt	Vorschlag
Kernbeitrag	Weiterentwicklung des eigenen ServiceForwardRouters zu einer wiederverwendbaren Multi-Service-Routing-Infrastruktur.
Zielsystem	Neues DeviceIntegrationPlugin hinter der bestehenden Client-/WebAPI-Grenze und vor den DeviceAgents.
MVP	Registrierung, NotifyDeviceList-Vollsnapshot, eine Duplex-Verbindung, Unicast-/Broadcast-Routing, TreeNode-Anbindung und automatisierte Tests.
Nicht-Ziel	Ersatz der DeviceAgents oder Vereinheitlichung ihrer Geräteprotokolle.

ENTSCHEIDUNGSVORSCHLAG Der vorhandene ServiceForwardRouter wird als eigene Vorarbeit und Ausgangsbasis behandelt. Die Bachelorarbeit umfasst seine Entkopplung, Generalisierung und Erweiterung sowie die prototypische Integration in das zenLAB-Framework.

1. Ausgangslage und Ziel

Die konkreten DeviceAgents bleiben für Hardwarekommunikation, Geräteprotokolle und TreeNode-Funktionen verantwortlich. Der AgentManager startet und überwacht nur noch deren Prozesse. Das im Showcase als mehrere Klassen vorhandene Service-Forward-Routing-Subsystem überträgt Aufträge und Antworten, unterstützt aber noch keine saubere Trennung mehrerer dynamisch registrierter Service-Sessions.

Das neue DeviceIntegrationPlugin schließt diese Lücke. Es verwaltet aktive Integrationen und ihre Devices, überwacht deren Lebenszyklus, hostet Client- und Agent-Service und vermittelt TreeNode-Aufträge sowie Ereignisse über genau eine bidirektionale Verbindung je Registrierung.

1.1 Forschungs- und Entwicklungsfrage

LEITFRAGE Wie kann der bestehende ServiceForwardRouter zu einer wiederverwendbaren Multi-Service-Routing-Infrastruktur weiterentwickelt werden, die mehrere dynamisch registrierte DeviceAgents eindeutig adressiert und deren TreeNode-Funktionen für Clients bereitstellt?

1.2 Umfang

Im MVP enthalten	Optional / später
------------------	-------------------

Im MVP enthalten	Optional / später
Integration registrieren und eindeutig identifizieren	Produktionsreife bidirektionale Gateway-Laufzeit
vollständige Device-Liste als Lease-Signal verarbeiten	Feingranulare Autorisierung und Security-Härtung
Unicast- und Broadcast-Aufträge routen und Ergebnisse aggregieren	Persistierte oder verteilte Registry
TreeNode-Aufrufe und Node-Streams einschließlich ChildNodeAdded/Removed ausführen	Produktionsreifer Umbau des ASP.NET Core Adapters
Timeout, Streamverlust, Neuregistrierung und Deregistrierung behandeln	Migration sämtlicher vorhandener DeviceAgents
SmartProxy und gRPC-Servicevertrag anbinden	Änderungen am TreeNodeBasedClient
Unit- und Integrationstests	Umfangreiche Langzeit- und Lasttests

2. Vorgeschlagene Architektur

Zielarchitektur mit Client-/Agent-Service, Node-Streams und einer Duplex-Verbindung

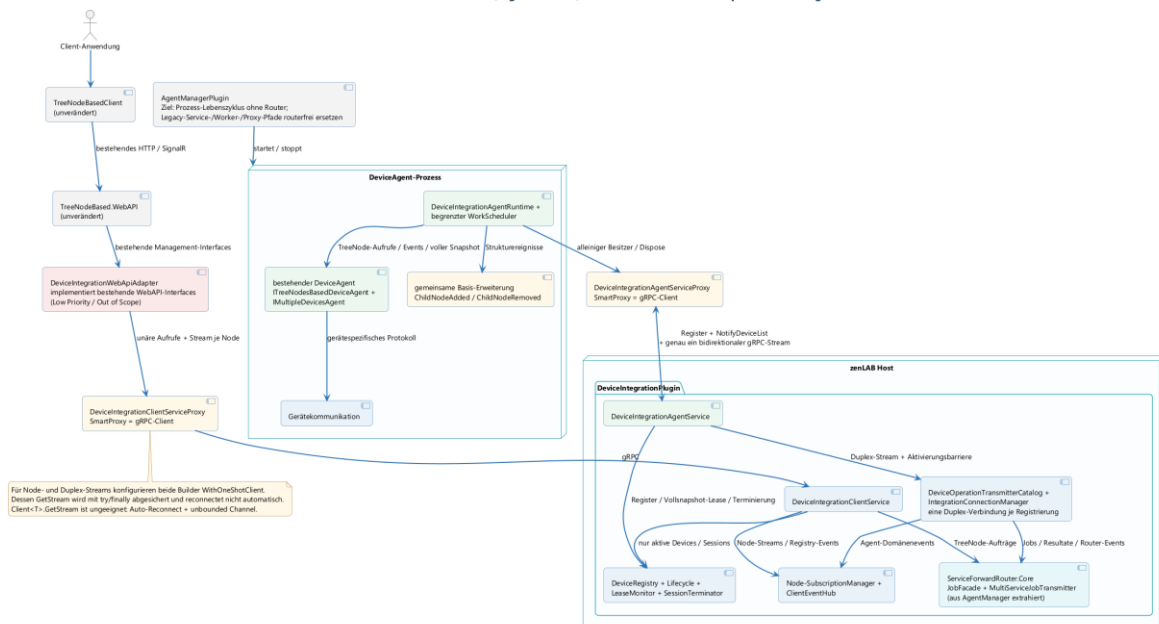


Abbildung 1: Vorgeschlagene Komponentenarchitektur (PlantUML)

Analog zum bestehenden DeviceManagementPlugin werden zwei getrennte code-first-gRPC-Servicegrenzen verwendet: eine für Client-Aufrufe und eine für Agents beziehungsweise Integrationen. Der jeweilige SmartProxy ist der konkrete gRPC-Client und implementiert denselben Vertrag wie der serverseitige Service. Für langlebige Node- und Duplex-Streams konfigurieren beide Builder ausdrücklich WithOneShotClient. Der vorhandene Client<T>.GetStream ist wegen automatischem Reconnect und unbounded Zwischen-Channel dafür ungeeignet; OneShotClient.GetStream wird deshalb ohne Retry und mit try/finally für den gRPC-Channel abgesichert.

Komponente	Verantwortung
TreeNodeBasedClient	Bleibt unverändert und verwendet weiterhin die bestehende HTTP-/SignalR-Schnittstelle.
ASP.NET Core Adapter	Bestehende WebAPI-Grenze; Anpassung ist Low Priority und nicht Teil des Kernumfangs.
DeviceIntegrationClientServiceProxy	SmartProxy und konkreter gRPC-Client der WebAPI; implementiert IDeviceIntegrationClientService.
DeviceIntegrationClientService	Serverseitiger gRPC-Service im Plugin für TreeNode-Aufrufe, einen Stream je konkreter Node-Subscription sowie einen getrennten Registry-Eventstream.
DeviceIntegrationAgentServiceProxy	SmartProxy und konkreter gRPC-Client im DeviceAgent-Prozess; implementiert IDeviceIntegrationAgentService.
DeviceIntegrationAgentService	Serverseitiger gRPC-Service im Plugin für Registrierung, vollständige Device-Listen, Abmeldung und genau einen bidirektionalen Stream für Jobs, Resultate und Events.
Device Registry / Lease Monitor	Verwaltet Device-zu-ServiceSession-Zuordnung, Registrierungen und die durch NotifyDeviceList erneuerte Lease.
Subscription Manager	Bündelt identische Subscriptions und verteilt NodeChanges an Clients.
ServiceForwardRouter	Bezeichnet das Subsystem aus JobFacade, JobTransmitter, JobRequestFactory und Hilfsklassen; es wird vollständig aus AgentManager herausgelöst und um Multi-Service-Routing erweitert.
Integration Adapter	Orchestriert AgentServiceProxy, Duplex-Stream, WorkRunner, EventForwarder und die vorhandenen Interfaces ITreeNodeBasedDeviceAgent sowie IMultipleDevicesAgent.

3. Hauptablauf und Contracts

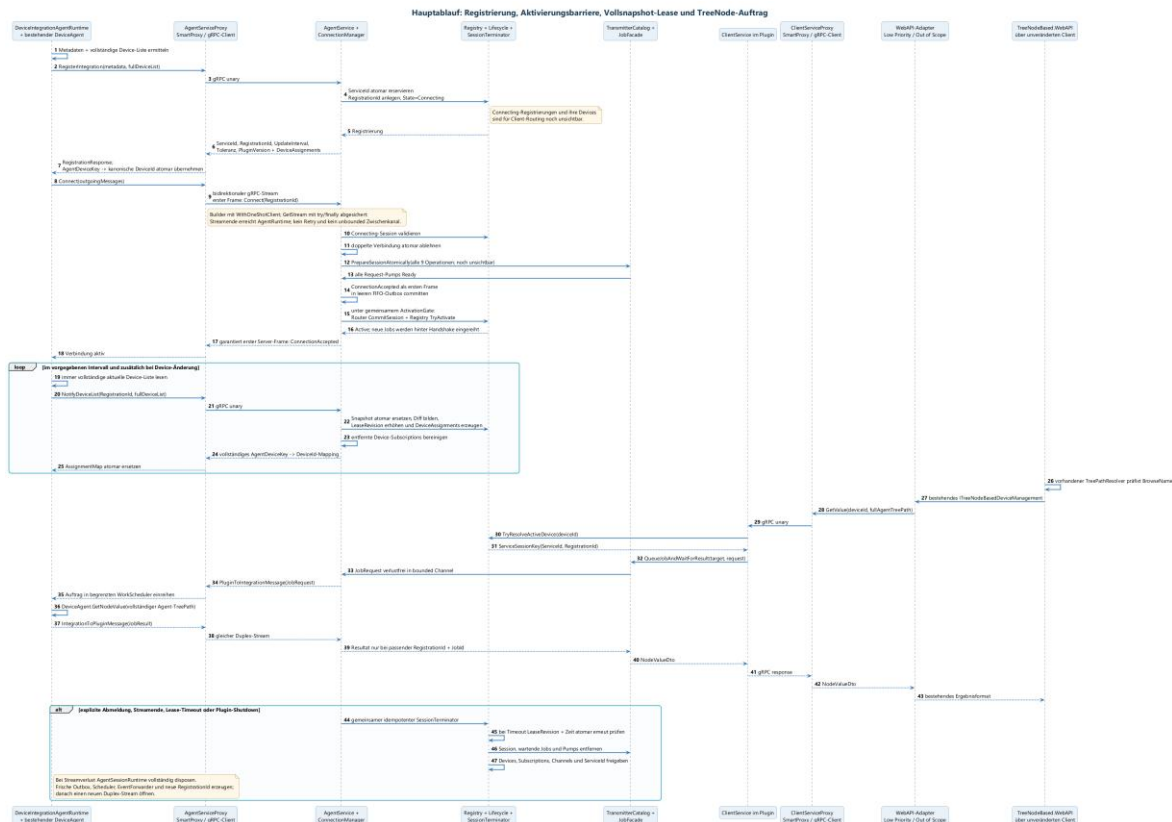


Abbildung 2: TreeNode-Auftrag über Client- und Agent-ServiceProxy (PlantUML)

3.1 Registrierung und Lebenszyklus

- Der DeviceIntegrationAgentRuntime registriert sich über den DeviceIntegrationAgentServiceProxy beim DeviceIntegrationAgentService und sendet RequestedServiceId, Metadaten sowie die vollständige aktuelle Device-Liste. Jeder Snapshot-Eintrag trägt einen im Agent-Prozess stabilen AgentDeviceKey.
- Der DeviceIntegrationAgentService validiert und reserviert die ServiceId, erzeugt eine RegistrationId und hält die Registrierung zunächst im Zustand Connecting. Erst wenn die Duplex-Verbindung eindeutig angenommen und alle neun Request-Pumps bereit sind, wird sie atomar auf Active gesetzt und für Clients sichtbar.
- NotifyDeviceList enthält immer die vollständige aktuelle Device-Liste, ersetzt den Snapshot atomar und erneuert zugleich die Lease. Die Antwort liefert jedes Mal die vollständige Zuordnung AgentDeviceKey zu kanonischer DeviceId; der Agent ersetzt seine bidirektionale Zuordnung atomar. Ein separates Agent-KeepAlive gibt es nicht.
- Bleibt NotifyDeviceList länger als UpdateInterval plus Tolerance aus, prüft die Registry LeaseRevision und Zeitstempel erneut atomar. Nur dann werden Verbindung, wartende Jobs, Devices und Subscriptions über denselben idempotenten SessionTerminator entfernt.
- Jeder Frame und jeder Cleanup prüft die RegistrationId, damit ein alter Stream oder Timeout niemals eine neuere Verbindung mit gleicher ServiceId verändert.

3.2 Vorgeschlagene Job-Envelopes

```
public readonly record struct ServiceSessionKey(
    string ServiceId, Guid RegistrationId);

public enum JobResultKind { Response, Event }

public sealed class JobRequest<T>
{
    public ulong JobId { get; init; }
    public ServiceSessionKey Target { get; init; }
    public string SerializedParameter { get; init; }
    [IgnoreDataMember] public T? Parameter { get; init; }
}

public sealed class JobResult<T>
{
    public ulong JobId { get; init; }
    public ServiceSessionKey Source { get; init; }
    public JobResultKind Kind { get; init; }
    public bool Succeeded { get; init; }
    public string SerializedValue { get; init; }
    public ExceptionDto? Exception { get; init; }
    [IgnoreDataMember] public T? Value { get; init; }
}
```

3.3 Routingregeln

Situation	Verhalten
Ziel aktiv	Auftrag an den Duplex-Stream der aktuellen Registrierung weiterleiten.
Ziel unbekannt oder inaktiv	Typisierten ServiceUnavailable-Fehler liefern.
Antwort passt nicht zu JobId/Service/RegistrationId	Antwort verwerfen und Diagnoseereignis schreiben.
Timeout	Warten abbrechen und Target, JobId und Deadline im Fehler bereitstellen.
Streamverlust / Neuregistrierung	Neue RegistrationId; verspätete Resultate der alten Verbindung sind ungültig.

4. Kernmodell und Projektstruktur

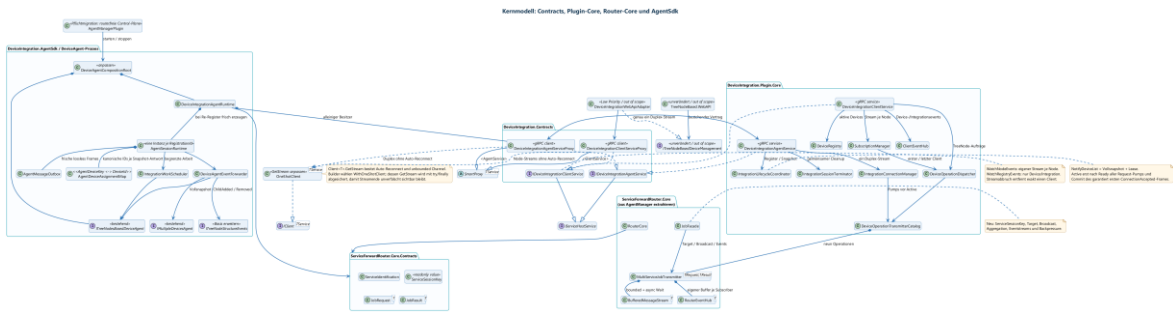


Abbildung 3: Getrennte Service-/Proxy-Paare für Client- und Agent-Seite (PlantUML)

4.1 Empfohlene Projekte

Projekt	Inhalt
ServiceForwardRouter.Core.Contracts	Allgemeine Job-, Result-, Routing- und Stream-Contracts.
ServiceForwardRouter.Core	JobFacade, async Transmitter, ServiceSession-Routing, Target, Broadcast, Aggregation und Eventstreams.
DeviceIntegration.Contracts	Client- und Agent-Serviceverträge, DTOs, SmartProxys, gemeinsamer MessageCodec sowie Builder mit expliziter OneShot-Konfiguration für langlebige Streams.
DeviceIntegration.Plugin.Core	Registry, Lifecycle, SessionTerminator, Connections, Dispatcher, Services und Subscription-Verwaltung.
DeviceIntegration.AgentSdk	AgentRuntime, SmartProxy und ein frischer AgentSessionRuntime je RegistrationId mit eigener Outbox, Cancellation, WorkScheduler, DeviceId-Zuordnung sowie Snapshot- und Event-Bridge.
DeviceIntegration.Tests	Proxy-, Service-, Contract-, Routing-, Integrations- und Nebenläufigkeitstests.

4.2 Abhängigkeiten

- Der `TreeNodeBasedClient` bleibt unverändert und kommuniziert weiterhin mit der `TreeNodeBased.WebAPI`.
- Die bestehende WebAPI verwendet weiterhin `IMediator`, `ITreeNodeBasedDeviceManagement` und `IDeviceCollectionContainer`. Ein späterer, ausdrücklich als Low Priority markierter Adapter implementiert diese vorhandenen Interfaces und verwendet intern den `DeviceIntegrationClientServiceProxy`.
- Der `DeviceIntegrationAgentRuntime` besitzt und disposed den `DeviceIntegrationAgentServiceProxy`. Er registriert sich, öffnet genau einen bidirektionalen Stream, multiplexed darin Jobs, Resultate und Events und meldet parallel vollständige Device-Snapshots.
- `DeviceIntegrationClientService` und `DeviceIntegrationAgentService` laufen im Plugin. Sie greifen ausschließlich über Lifecycle-, Dispatcher-, Connection- und Subscription-Abstraktionen auf die gemeinsam als SingleInstance registrierten Zustände zu. Weil ein vorhandener DeviceAgent SubscriptionCreated bereits synchron vor seiner MethodResponse melden kann, puffert der SubscriptionManager diese Reihenfolge im Pending-Zustand begrenzt und gibt an Clients stets Accepted oder Rejected vor Created aus.
- Gerätespezifische Protokolle und TreeNode-Logik bleiben in den bestehenden DeviceAgents. Die gemeinsame Management-Basis beziehungsweise der DeviceAgent-Composition-Root muss jedoch vom alten DeviceManagementPlugin entkoppelt werden.

5. Umsetzungsvorschlag

Phase	Arbeitspaket	Ergebnis
0	Ausgangsstand sichern	Zuerst die begonnene Solution buildfähig und als Plugin auffindbar machen: GlobalAssemblyInfo-Pfad korrigieren sowie MEF-Export und PluginMetadata ergänzen. Danach den vorhandenen Router mit Characterization Tests absichern und seine Abhängigkeiten dokumentieren.
1	Router extrahieren	Routing- und Contracts-Projekte in den DeviceIntegrationPlugin-Bereich verschieben; alte Router-, Proxy-, Service- und Worker-Verweise aus AgentManager entfernen.
2	Multi-Service-Routing	ServiceIdentification, ServiceSessionKey, MultiServiceJobTransmitter, Zielrouting, Broadcast und Aggregation implementieren.
3	gRPC-Services und Registrierung	Client-/Agent-Serviceverträge, beide SmartProxys, OneShot-Streaming, atomare Aktivierungsbarriere, Register, Vollsnapshot-Lease und idempotentes Cleanup umsetzen.
4	Integration Client / Adapter	Genau eine Duplex-Verbindung, MessageCodec, WorkRunner und Anbindung an ITreeNodesBasedDeviceAgent/IMultipleDevicesAgent implementieren.
5	Subscriptions	Zusammengesetzte ClientSubscriptionKeys, RefCount, geordnete Accepted/Rejected-Ereignisse, per-Client-Fan-out und echte begrenzte Backpressure umsetzen.
6	Evaluation	Referenz-Agent anbinden, Anforderungen testen und Routing-Overhead messen.

5.1 Teststrategie

Testebene	Prüfung
Unit	ID-Validierung, TreePathResolver, DeviceKey/DeviceId-Rückabbildung, atomare Lease-Grenzen, Korrelation, Subscription-RefCount und Ereignisreihenfolge.
Contract	Serialisierung und Kompatibilität der öffentlichen DTOs.
Integration	Mindestens zwei Integrationen und mehrere Clients; korrektes Unicast-Routing.
Nebenläufigkeit	NotifyDeviceList, Timeout, Streamende, vollständige Neuregistrierung und Unregister gleichzeitig.
Fehlerfälle	Service offline, Stream-Abbruch, falsches oder verspätetes Result.
Performance	Plugin-Routing-Overhead und Verhalten bei vielen logischen Subscriptions.

5.2 Definition of Done für den MVP

- Zwei Integrationen können sich gleichzeitig mit unterschiedlichen IDs registrieren.
- Doppelte IDs werden deterministisch abgelehnt.
- Ein Client-Auftrag erreicht ausschließlich die adressierte Integration.

- Antworten werden nur bei passender JobId, ServiceId und RegistrationId akzeptiert.
- Eine ausbleibende vollständige NotifyDeviceList-Meldung führt nach Intervall plus Toleranz genau einmal zum vollständigen Cleanup.
- Ein bestehender DeviceAgent wird ohne Änderung seiner Gerätekommunikation angebunden; nur gemeinsame Basis beziehungsweise Composition Root werden angepasst.
- Automatisierte Tests laufen reproduzierbar im Build.

6. Entscheidungen für das Betreuergespräch

Die folgenden Punkte sollten vor Implementierungsbeginn bestätigt werden. Sie beeinflussen Umfang, Forschungsfragen und Evaluation der Bachelorarbeit.

ID	Entscheidungsfrage	Vorschlag
D-01	Ist die Entkopplung und Multi-Service-Erweiterung des ServiceForwardRouters der zentrale Eigenbeitrag?	Ja; bestehender Router wird als eigene Vorarbeit beschrieben.
D-02	Gehören Subscriptions vollständig zum MVP?	ValueChanged, SubscriptionCreated/Disposed und ChildNodeAdded/Removed gehören laut Konzept zum MVP. Zusätzlich ist zu bestätigen, ob WatchNodeEvents die bisher getrennte Subscribe-Operation bewusst als langlebigen Aufruf bündelt.
D-03	Gehören Broadcast und Gateway zum Pflichtumfang?	Broadcast und Ergebnisaggregation gehören zum Router-Kern; das Gateway wird konzipiert und nur bei ausreichendem Zeitbudget implementiert.
D-04	Welcher DeviceAgent dient als Referenz?	Empfehlung: Simulator plus ein überschaubarer realer TreeNode-Agent.
D-05	Welche Rückwärtskompatibilität ist erforderlich?	AgentManager soll nach der Migration nur Prozessionsstart und -überwachung leisten. Dafür müssen die heute routergebundenen DeviceRunner-, DeviceProvider-, AgentProvider-, AgentInformationProvider- und ConfigurationEditor-Service-/Worker-/Proxy-Pfade entfernt oder routerfrei ersetzt werden.
D-06	Welche Leistungsziele werden bewertet?	Zielwerte für aktive Integrationen, Clients, Subscriptions und p95-Overhead festlegen.
D-07	Wird Security implementiert oder nur konzipiert?	Vorhandene Remoting-Security nutzen; feingranulare Autorisierung als Abgrenzung möglich.

6.1 Erwartete Ergebnisse

- Wiederverwendbare Router-Pakete und dokumentierte öffentliche Contracts.

- Lauffähiger MVP des DeviceIntegrationPlugins im zenLAB-Host.
- Integration Client mit Adapter für mindestens einen bestehenden DeviceAgent.
- Automatisierte Tests und nachvollziehbare Evaluationsdaten.
- PlantUML-Diagramme und Architekturentscheidungen für die Bachelorarbeit.

EMPFOHLENE FREIGABE Ein vertikaler Unicast-Durchstich ist der erste Zwischenmeilenstein. Zum MVP gehören danach Registrierung, Vollsnapshot-Lease, eine Duplex-Verbindung, Target/Broadcast/Aggregation und Node-Streams einschließlich ChildNodeAdded/Removed. Gateway und WebAPI-Adapter bleiben optionale Folgearbeiten.

7. Use-Case-Sichten

Die folgenden Use-Case-Sichten zeigen getrennt, was Integration Clients, Client-Anwendungen und der Betrieb mit dem Device-Integration-Plugin tun. Die bestehenden DeviceAgents bleiben dabei die Ausführungsziele für gerätespezifische Operationen.

Gesamtübersicht der Anwendungsfälle

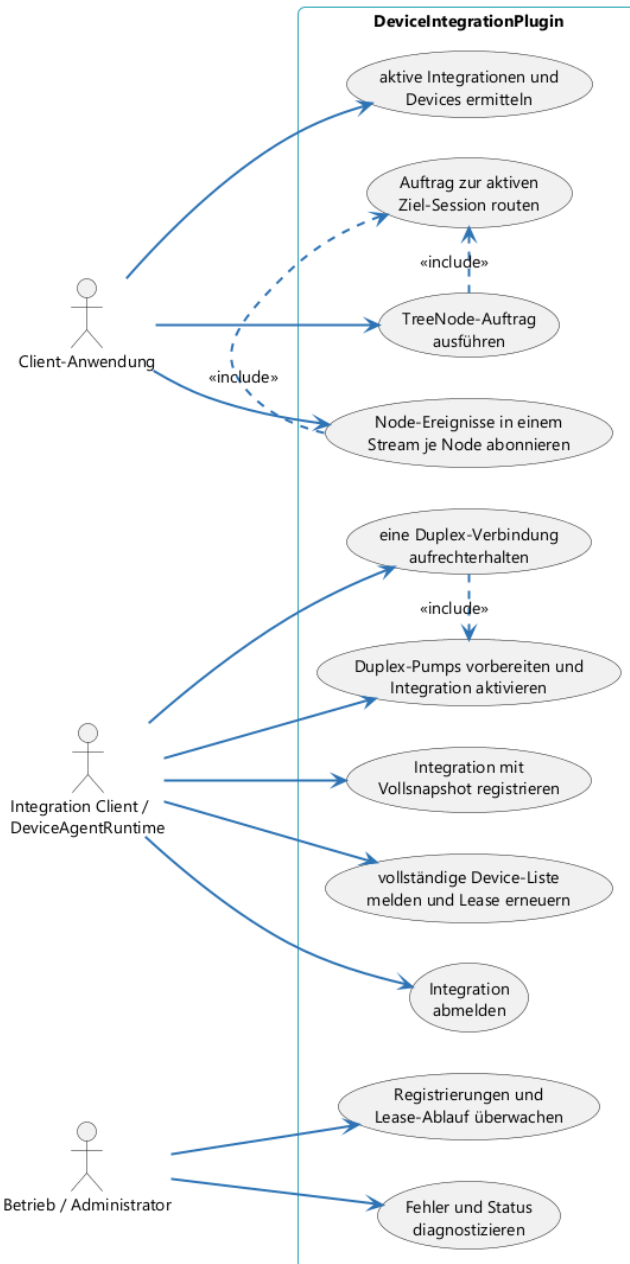


Abbildung 4: Gesamtübersicht der Akteure und Use Cases (PlantUML)

7.1 Lebenszyklus einer Integration

Diese Sicht beschreibt die Registrierung einer Integration, ihre Laufzeitüberwachung und das Aufräumen veralteter Zustände.

Lebenszyklus einer Integration

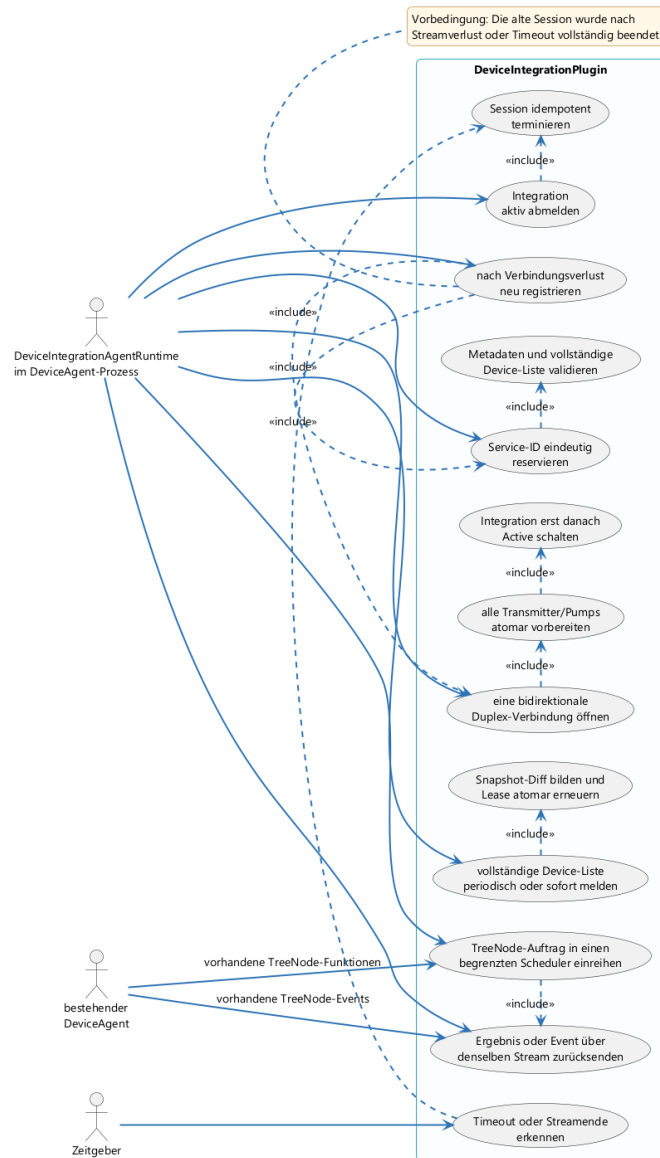


Abbildung 5: Registrierung, Aktivierungsbarriere, Vollsnapshot-Lease, Neuregistrierung und Cleanup (PlantUML)

7.2 Clientzugriff auf Devices

Diese Sicht beschreibt, wie eine Client-Anwendung über den unveränderten `TreeNodeBasedClient` auf Integrationen und Devices zugreift. Das Plugin löst `DeviceId` und aktive `ServiceSession` auf; der Auftrag erreicht den Agent über dessen bereits geöffneten bidirektionalen gRPC-Stream.

Clientzugriff auf Devices und Node-Streams

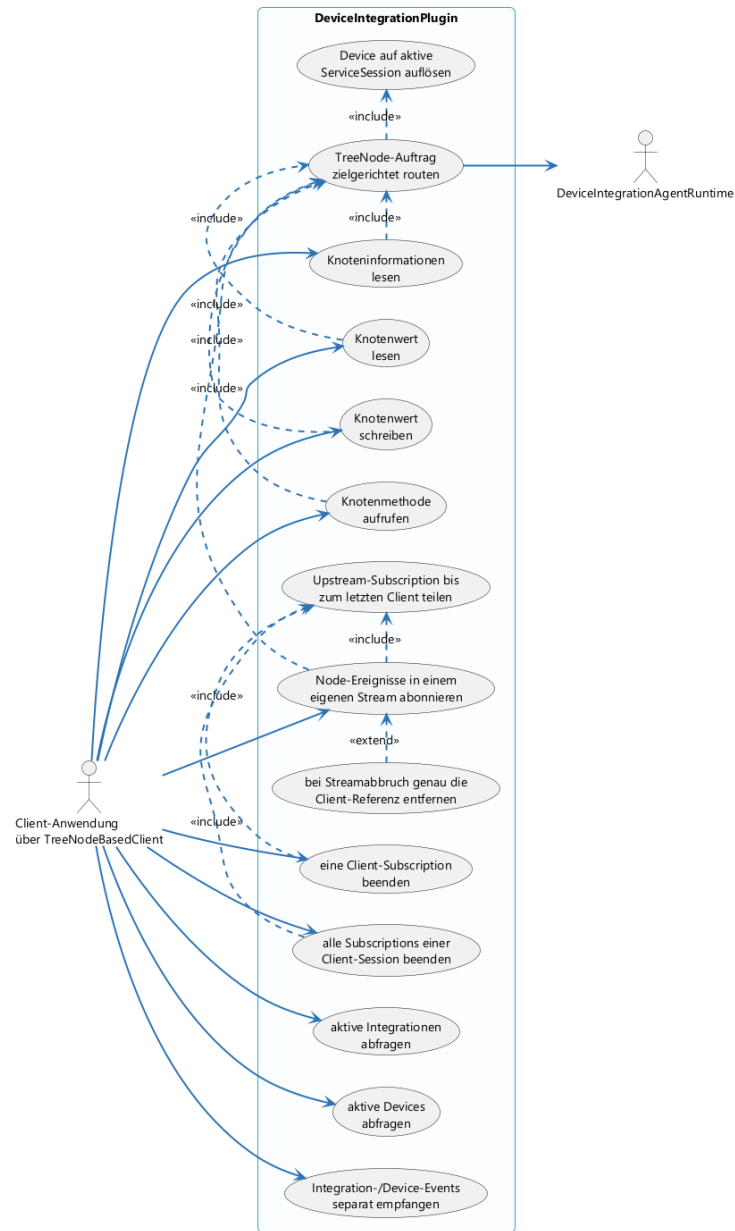


Abbildung 6: TreeNode-Aufrufe und Subscriptions aus Clientsicht (PlantUML)